Problem Statement: Text-to-Learn: AI-Powered Course Generator

Title: Text-to-Learn: AI-Powered Course Generator

Overview:
This hackathon challenges participants to build a full-stack web application capable of transforming any user-submitted topic into a structured, multi-module online course. The application will accept a free-form text prompt (e.g., "Intro to React Hooks," "Basics of Copyright Law") and automatically generate a complete course outline, detailed lesson content, and a resource list, providing an immediate, tailored learning experience.

Context & Objectives:
The primary purpose is to gain end-to-end full-stack development experience, specifically focusing on integrating AI or template-based content generation. Key learning goals include designing and implementing a prompt-ingestion API, integrating with AI/text-templating services (like OpenAI or Hugging Face), persisting generated course structures in a database, presenting course details via a clean and responsive UI, and deploying the application to a cloud platform with CI/CD.

Core Challenge:
Develop "Text-to-Learn," a web application that:

- **Input:** Accepts a free-form text prompt describing the desired learning topic.
- **Process:** Leverages an AI/text-generation endpoint or a custom algorithm to produce:
  - A comprehensive course title and description.
  - 3–6 modules, each containing 3–5 lessons.
  - For each lesson: clear objectives, key topics, and suggested readings or external links.
- **Output:** Renders a navigable course syllabus on the frontend and ensures persistent storage in the backend.

Deliverables:
Participants are required to submit a live, publicly accessible site URL, a GitHub repository with detailed commit history and a clear README, and a 5-minute video demonstrating the prompt input, course generation flow, and an overview of the code architecture.

Scope & Technology Stack:

- **Frontend:** React, Vue, Angular, or vanilla JS + HTML/CSS.
- **Backend:** MERN (Node.js + Express + MongoDB) is recommended, with alternatives like Spring Boot or Django permitted.
- **AI/Text-Gen:** OpenAI API, Hugging Face Inference, or a rule-based template engine.

- **Version Control:** GitHub, emphasizing feature branches and pull requests.
- **Deployment:** Heroku, Vercel, AWS, or similar platforms, with CI/CD pipelines encouraged.

Evaluation Criteria:
Projects will be assessed on functionality (correct prompt-to-course generation), code quality (architecture, modularity, documentation), design/UI (usability, responsiveness, visual polish), and creativity (novel features like PDF export or customizable templates).

Hackathon Roadmap: Text-to-Learning Course Generator

Milestone 1: Project Conception and Prototype Thinking

Goal: Establish the core concept of the application and define essential features and user flow from a user-centric perspective.

Problem Statement: Develop a full-stack, AI-powered web application that enables users to input a topic and receive a complete, structured online course. The application will enhance the learning experience with video content, PDF downloads, and multilingual support (e.g., Hindi explanations).

User Focus:

- **Purpose:** Simplify self-learning and course creation, offering immediate access to structured content.
- **Solutions:** Eliminate extensive searching, provide organized learning paths, and offer multilingual accessibility.
- **Enhancements:** Personalize learning through saved user sessions, enable offline consumption via PDF, and provide a structured approach to new topics.

Key Features Defined:

Feature	Description
Prompt to Course	Users input a topic prompt to generate structured modules and lessons.

Rich Lessons	Lessons incorporate text, code, video references, and quizzes.
Download as PDF	Individual lessons are downloadable in a formatted PDF.
Multilingual Explanations	AI-generated explanations in languages like Hinglish are included.
Secure Login	User authentication is handled via Auth0.
Persistent Courses	Users can save and revisit generated courses.

UI Flow & Components:

- **UI Flow:** The application will feature a home page (/) for course listing and creation, dedicated authentication pages (/login, /signup), a course overview page (/course/:id) displaying modules, and a detailed lesson viewer (/lesson/:id) with download options.
- **Key UI Components:**
  - **PromptForm:** Captures user topic input for course generation.
  - **SidebarNavigation:** Provides primary navigation links.
  - **LessonRenderer:** Displays structured lesson content.
  - **DownloadButton:** Initiates PDF download for lessons.

Project Significance: This project streamlines self-learning, accelerates access to organized educational content, broadens accessibility through multilingual support, facilitates offline learning, and offers a personalized user experience.

Milestone 2: Backend Setup

Goal: Establish a modular Node.js and Express backend to manage core application functionalities including course generation, user management, and data persistence.

Folder Structure Plan:

```
server.js              // Application entry point
/routes                // Defines API endpoints
/controllers           // Contains business logic for routes
/middlewares           // Handles authentication, error handling, and validation
/models                // Defines Mongoose schemas for database entities
/services              // Manages external logic, such as AI integrations
/utils                 // Houses utility functions (e.g., token generation)
/config                // Manages database connection and environment setup
```

Milestone 3: Frontend Setup

Goal: Configure a fast, modular, and scalable React frontend using modern build tools and a structured folder layout.

Technology Stack:

Tool/Library	Purpose
Vite	Provides a rapid development environment and bundler.
React	Serves as the UI framework.
UI Framework (e.g., Chakra UI, Tailwind CSS)	Ensures consistent and reusable components.
React Router DOM	Manages page-level navigation and routing.
State Management (e.g., Context API, Zustand)	Optional, for global application state management.

Generic Folder Structure:

```
/src
├── components/        // Reusable UI components (e.g., Sidebar, Forms)
```

```
├── pages/            // Page-level views (e.g., Home, Course, Lesson)
├── hooks/            // Custom React hooks (e.g., useAuth)
├── context/          // Context providers for global state
├── utils/           // Utility functions (e.g., API helpers)
├── App.jsx           // Root application file with routing logic
├── main.jsx          // Vite entry point
└── index.css         // Global styles (CSS reset or framework configuration)
```

Setup Highlights: The setup prioritizes performance with Vite, rapid component development with a UI framework, robust navigation with React Router, clear separation of concerns, and readiness for future scalability features like authentication and theming.

Milestone 4: Authentication (Auth0 Integration)

Goal: Integrate a secure OAuth 2.0-based authentication system using Auth0 to enable user login via redirects and secure backend routes with token verification.

Key Concepts:

Area	Purpose
Frontend (React SDK)	Manages login/logout flows, token storage, and user context.
Backend (Express Middleware)	Verifies access tokens and injects user data into requests.
OAuth2 Flow	Handles redirect-based login with consent screens and token exchange.
Access Token	A JWT token used in the Authorization header for authenticated API requests.

Implementation Details:

- **Frontend (React):** Utilize the Auth0 React SDK to wrap the application with Auth0Provider, implement loginWithRedirect() and logout(), access user state and tokens via useAuth0(), and enable silent token refresh for persistent sessions. Protect routes by checking authentication status.

- **Backend (Express):** Employ Auth0's JWT validation middleware to verify Bearer <access_token> in Authorization headers, fetch user information from Auth0's /userinfo endpoint, and attach user metadata (email, name, sub) to req.user for subsequent use in authenticated API routes (e.g., /api/save-course, /api/user-courses).

Integration Checklist:

1. Register the application in the Auth0 Dashboard (Client ID, Domain, Audience).
2. Configure the frontend provider with the correct domain, client ID, and redirect URI.
3. Develop login/logout UI using the SDK.
4. Include access tokens in API requests to authenticated backend routes.
5. Validate tokens and attach user information on backend APIs.

Outcome: Successful integration will result in a seamless login/logout experience, secured backend APIs, user-specific access to courses and progress, and the ability to associate saved content with individual user accounts.

Milestone 5: Database Schema Design

Goal: Design and implement MongoDB data models using Mongoose to create a clean, scalable structure for storing and relating users, courses, modules, and lessons.

Core Entities & Relationships:

Entity	Description
User	An Auth0-identified user capable of creating and managing courses.
Course	A comprehensive course composed of multiple modules.
Module	A distinct section within a course, containing several lessons.
Lesson	A singular unit of content, comprising rich, structured learning material.

Schema Overview (Mongoose):

- **Course Schema:** Defines a course with a title, description, creator (Auth0 sub), references to associated modules, and tags for categorization. It supports a one-to-many relationship with modules.

```JavaScript
const courseSchema = new mongoose.Schema({
 title: { type: String, required: true },
 description: String,
 creator: { type: String, required: true }, // typically Auth0 `sub`
 modules: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Module' }],
 tags: [{ type: String, trim: true }]
}, { timestamps: true });
```

- **Module Schema:** Defines a module with a title, a required reference to its course, and an array of references to its lessons. Each module belongs to a single course.

```JavaScript
const moduleSchema = new mongoose.Schema({
 title: { type: String, required: true },
 course: { type: mongoose.Schema.Types.ObjectId, ref: 'Course', required: true },
 lessons: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Lesson' }]
}, { timestamps: true });
```

- **Lesson Schema:** Defines a lesson with a title, a flexible content array for structured blocks (e.g., paragraphs, code, quizzes), an isEnriched flag to track AI enhancements, and a reference to its parent module.

```JavaScript
const lessonSchema = new mongoose.Schema({
 title: { type: String, required: true },
 content: { type: [mongoose.Schema.Types.Mixed], required: true }, // flexible structured blocks
 isEnriched: { type: Boolean, default: false }, // to track if AI-enhanced
 module: { type: mongoose.Schema.Types.ObjectId, ref: "Module" }
});
```

Milestone 6: Lesson Rendering (Frontend Components)

Goal: Implement the rendering of structured lesson content from JSON data using modular React components.

Content Format Example:

JSON

```
[
 { "type": "heading", "text": "Introduction to AI" },
 { "type": "paragraph", "text": "Artificial intelligence (AI) is a rapidly evolving field..." },
 { "type": "code", "language": "python", "text": "print('Hello, AI!')" },
 { "type": "video", "url": "AI Explained" },
 { "type": "mcq", "question": "What is AI?", "options": ["A type of robot", "A field of computer science", "A programming language"], "answer": 1 }
]
```

Components:

- **LessonRenderer:** This component will iterate through the content array and dynamically render specific block components based on the type field.
- **Block Components:** Individual components for each content type will be developed, including HeadingBlock, ParagraphBlock, CodeBlock, VideoBlock, and MCQBlock.

Folder Structure Suggestion:

```
/src
├── components/
|   └── LessonRenderer.jsx
|   └── blocks/
|       ├── HeadingBlock.jsx
|       ├── ParagraphBlock.jsx
|       ├── CodeBlock.jsx
|       ├── VideoBlock.jsx
|       └── MCQBlock.jsx
```

Milestone 7: React Router and Sidebar Layout

Goal: Establish route-based navigation with a consistent application layout (sidebar, topbar) using React Router v6.

Key Routes:

- /: The primary application layout with a sidebar and nested views.
- index: The default home view within the main layout.
- /courses/:courseId/module/:moduleIndex/lesson/:lessonIndex: The route for viewing specific lessons.

Setup Summary: App.jsx will manage the overall layout using <Outlet /> for rendering nested pages. A persistent **Sidebar** component will be integrated within App.jsx. Pages will be rendered via <Routes> in main.jsx (or a dedicated routing file). Authentication pages will be handled separately by Auth0's redirect flow.

Frontend UI Emphasis (New/Re-emphasized):

- **Error Handling and Loading States:** Implement global loading indicators (e.g., for AI generation, API calls) and user-friendly error messages for API failures or invalid inputs. This includes dedicated LoadingSpinner and ErrorMessage components.

Milestone 8: AI Prompt Design (Course and Lesson Generation)

Goal: Design structured prompts for the AI to effectively generate both full course outlines (modules and lesson titles) and detailed lesson content in a strict JSON format.

Prompt Strategy:

1. **generateCoursePrompt(topic):**
    - **Input:** A topic string (e.g., "Introduction to Machine Learning").
    - **Output:** A JSON object containing title, description, tags, and an array of modules, each with an array of lesson titles.
    - **Logic:** The prompt will guide the AI to generate a curriculum that progresses from foundational to advanced concepts, ensuring comprehensive coverage of essential subtopics.
2. **generateLessonPrompt(course, module, lesson):**
    - **Input:** Titles of the course, module, and lesson (e.g., "Introduction to Machine Learning", "Basics of ML", "What is Supervised Learning?").
    - **Output:** A JSON object with the title, objectives (newly added), and a content array (matching the format from Milestone 6) comprising various block types like headings, paragraphs, code, video, and MCQs.
    - **Rules:**
        - Include a **structured field for lesson objectives** (e.g., objectives: ["Understand...",

"Identify..."]).
- Include a **video search query** instead of direct links.
- Incorporate a **code block** only when relevant to the lesson content.
- Add **4–5 multiple-choice questions (MCQs)** at the end of the lesson, including an explanation for the correct answer.
- The AI must return **raw JSON only**, without Markdown formatting or additional explanatory text.

Milestone 9: Video Generation (YouTube API Integration)

Goal: Integrate the YouTube Data API v3 to fetch relevant educational videos for each lesson based on AI-generated video search queries.

Strategy:

- Extract the { type: "video", query: "..." } object from the lesson JSON.
- Utilize the **YouTube API's search endpoint** to retrieve top video results, specifying parameters like q (query), maxResults (1–3), type=video, and videoEmbeddable=true.

Implementation:

- The **VideoBlock.jsx** component will accept a query prop.
- It will make a backend API call to /api/youtube?query=..., which will return the video ID or embed URL.
- The component will then render a YouTube iframe using the retrieved video information.

Considerations:

- A **YouTube Data API key** is required and must be kept secure.
- Caching mechanisms can be implemented to reduce API quota usage.

Milestone 10: Multilingual Lesson Explanation (Hinglish)

Goal: Provide students with Hinglish audio explanations of lesson content to enhance accessibility.

Strategy:

- Leverage the **Gemini API** for:
    - **Translation:** Converting English lesson text to Hinglish.

- **Text-to-Speech (TTS):** Generating audio from the translated text.

Workflow:

1. **Input:** The English lesson text will serve as the input.
2. **Translation:** If enabled (e.g., autoTranslateToHindi flag or user request), the text will be translated into Hinglish.
3. **Audio Generation:** The Gemini TTS model will generate audio from the translated text.
4. **Output:** A .wav buffer will be returned to the frontend for immediate playback or download.

Notes:

- The TTS service supports various voiceName options.
- Translations are contextually aware and designed to assist students with partial English fluency.
- Future enhancements could include a language toggle for additional options (e.g., pure Hindi, Tamil).

Milestone 11: Download Lesson as PDF

Goal: Enable users to download complete lesson content (text, code, MCQs) as a styled PDF document.

Strategy:

- Utilize **html2canvas** to capture the DOM element of the lesson as an image.
- Employ **jsPDF** to convert this captured image into a downloadable PDF.
- Apply custom styling, including dark theme support, specific fonts, layout, and code block formatting, to ensure a polished PDF appearance.

Core Component:

- **LessonPDFExporter.jsx:** This component will include a download button. It will use a hidden lesson rendering section (managed by a React ref), which will be specifically styled for PDF export to maintain consistent formatting irrespective of the active UI theme. Custom styles will be injected to optimize the PDF's visual presentation.

Additional Features:

- The exporter will correctly handle formatting elements such as bold text, newlines, and code blocks.
- The functionality can be extended to export individual modules or entire courses.

Milestone 12: Hosting and Deployment

Goal: Deploy the backend on Render and the frontend on Vercel, integrating them for seamless operation, and establish a robust CI/CD pipeline.

Backend Hosting on Render

1. **Push Backend to GitHub:** Ensure your backend repository (text-to-learn-backend) is structured correctly with the /server directory at the root (or specify /server as the root directory on Render).
2. **Create Web Service on Render:**
    - Navigate to Render and select "New Web Service".
    - Connect your GitHub account and choose the text-to-learn-backend repository.
3. **Configure Service:**
    - **Build Command:** npm install
    - **Start Command:** node server.js (or npm run start)
    - **Root Directory:** Specify /server if your main project repository contains both client and server directories; otherwise, leave empty.
4. **Add Environment Variables:** In your Render dashboard under the Environment tab, set the following:
    - PORT=5000
    - MONGO_URI=your_mongodb_connection_string
    - AUTH0_ISSUER=https://your-auth0-domain.auth0.com/
    - AUTH0_AUDIENCE=your-api-identifier
    - GEMINI_API_KEY=your-google-genai-key
    - YOUTUBE_API_KEY=your-youtube-data-api-key

Result: Your backend will be live at a URL like https://text-to-learn-backend.onrender.com.

Frontend Hosting on Vercel

1. **Push Frontend to GitHub:** Ensure your frontend repository (text-to-learn-frontend) has its /client directory at the root (or specify /client as the root directory on Vercel).
2. **Import Project on Vercel:**
    - Select "Add New Project".
    - Choose your text-to-learn-frontend GitHub repository.
    - Specify /client as the root directory if your main project repository contains both client and server directories.

3. **Set Environment Variables:** In Vercel Project Settings under Environment Variables, add:
    - VITE_AUTH0_DOMAIN=your-auth0-domain.auth0.com
    - VITE_AUTH0_CLIENT_ID=your-auth0-client-id
    - VITE_API_URL=https://text-to-learn-backend.onrender.com (Use your actual Render backend URL)
    - VITE_YOUTUBE_API_KEY=your-youtube-data-api-key
4. **Update Frontend API Usage:** In your frontend code (e.g., client/utils/api.js), update API calls to reference the environment variable:

```
JavaScript
const API = import.meta.env.VITE_API_URL;
axios.post(`${API}/api/generate-lesson`, { ... });

// Or:
export const API_BASE_URL = import.meta.env.VITE_API_URL;
```

CI/CD Implementation:

- **Version Control & Workflow:** Establish a robust **Git workflow** from the outset, utilizing feature branches and Pull Requests (PRs) for organized development and code review, even in individual projects.
- **Automated Deployment:** Configure **CI/CD pipelines** (e.g., GitHub Actions) for both frontend and backend repositories. This will automate the build, test, and deployment processes, ensuring consistent builds, faster iterations, and a streamlined release cycle.

Final Folder Structure (Overall Project Repository):

```
project-root/
├── server/  // Deployed to Render
└── client/  // Deployed to Vercel
```

Milestone 13: Project Showcase and Documentation

Objective: Effectively communicate the project's impact, technical depth, and real-world relevance to potential employers, interviewers, and the broader developer community.

Resume Integration:

- **Project Title: Text to Learn Course Generator**
- **Live Demo & Code:**

    - **GitHub Repository:** [Link to your GitHub repo]
    - **Live Demo (Frontend):** [Sample Live Link Frontend: https://gpt-frontend.vercel.app/]
    - **Live Demo (Backend APIs):** [https://backendttl.onrender.com
    -]
- **Sample Resume Bullets:**
- Developed a full-stack AI course creation platform that generates structured learning modules and lessons from a user-provided topic prompt using large language models like Gemini/GPT.
- Implemented a rich lesson rendering system incorporating headings, paragraphs, multiple-choice questions (MCQs), code blocks, and dynamic video suggestions, providing an engaging and modular learning experience.
- Integrated multilingual text-to-speech (TTS) support, enabling Hinglish lesson narration through Google Gemini's translation and TTS APIs for enhanced accessibility.
- Designed and implemented PDF lesson export functionality using jsPDF and html2canvas, allowing users to download and access lessons offline.
- Secured the application with Auth0, providing authenticated access to personalized courses and enabling progress tracking for individual users.
- Utilized **React (Vite + Chakra UI)** for frontend development, **Node.js + Express + MongoDB** for the backend, and deployed on **Vercel** (frontend) and **Render** (backend) to ensure scalable cloud hosting.
- **Orchestrated a multi-stage AI prompt engineering pipeline** to sequentially generate structured course outlines and detailed lesson content, ensuring logical progression and comprehensive subject coverage.
- **Implemented robust parsing and validation of AI-generated JSON content** to maintain data integrity and consistency across rich lesson blocks (text, code, video queries, MCQs).
- Developed strategies to **minimize AI token usage and optimize response times** for content generation, balancing quality with computational efficiency.
- **Engineered dynamic UI components** in React to responsively render diverse content types (e.g., interactive quizzes, syntax-highlighted code blocks) from a flexible JSON structure.
- **Designed and implemented a scalable RESTful API** to manage CRUD operations for courses, modules, and lessons, facilitating seamless interaction between the frontend and database.
- **Modeled complex hierarchical data relationships** (Course > Module > Lesson) within MongoDB using Mongoose schemas, enabling efficient storage, retrieval, and manipulation of structured course data.
- **Implemented custom Express middleware** for API route protection, input validation, and centralized error handling, contributing to a secure and resilient backend.
- Maintained a **clean, modular codebase** following best practices for maintainability and future extensibility, evidenced by organized folder structures and clear component responsibilities.